

Simulation and Analysis of Buffer Manager Strategies with a Natural Language Query Interface

Team Name: i_am_groot

Team

- Aditya Prakash (23CS10088)
- Kaustav Mishra (23CS10030)
- Rohit Ranjeet Satpute (23CS10060)
- Tanishq Sura (23CS10071)
- Tanmay Nitin Amritkar (23CS30066)

Abstract

- Database systems rely on buffer managers to reduce expensive disk I/O operations.
- The buffer manager maintains a limited pool of memory pages and decides which disk pages remain cached.
- Query performance varies depending on the replacement strategy used.
- This project builds a simulation framework to analyze different buffer replacement policies.
- Queries will be given in natural language and translated into SQL using a language model.
- The generated SQL queries will be validated and executed on a database.
- Page access sequences obtained from executing SQL workloads (selection and join queries) will be fed into a C++ buffer manager simulator.
- Replacement policies such as LRU, MRU, and CLOCK will be implemented.
- Performance metrics such as disk reads, buffer hits, and hit ratios will be analyzed.

Motivation

- Disk access is significantly slower than memory access.
- Efficient buffering helps reduce disk I/O operations.
- Different replacement policies behave differently for different workloads.
- Studying these behaviors helps understand database system performance.
- Natural language interfaces make databases accessible to users without SQL knowledge.
- Combining natural language queries with a simulation framework provides both usability and educational insights into DBMS internals.

System Architecture

The system will consist of the following components:

- **Frontend Interface**
 - Accepts user queries written in natural language.
 - Implemented as either a command-line interface or a lightweight web interface.
- **LLM Query Translator**
 - Converts natural language queries into SQL queries.
 - Implemented using a Python-based language model interface.
- **Validation Pipeline**
 - Performs SQL syntax validation.
 - Ensures schema correctness.
 - Rejects queries referencing invalid tables or attributes.
- **Database Execution Layer**
 - Executes validated SQL queries on a relational database (SQLite/PostgreSQL).
 - Produces page access sequences during query execution.
- **Buffer Manager Simulator**
 - Implemented in C++.
 - Maintains a fixed-size buffer pool.
 - Simulates replacement strategies: LRU, MRU, CLOCK.
 - Tracks disk reads, page faults, and buffer hit ratios.

Implementation Plan

- Design relational database schema and sample datasets.
- Build a frontend interface to accept natural language queries.
- Implement the natural language to SQL translation module.
- Develop a validation pipeline for SQL queries.
- Execute SQL queries on a relational database.
- Generate page access traces from query execution.
- Implement the buffer pool simulation in C++.
- Implement replacement strategies: LRU, MRU, CLOCK.
- Track performance metrics such as disk I/O operations and hit ratios.
- Perform experiments under different workloads and buffer sizes.

Tech Stack

- **Languages**
 - C++ (Buffer Manager Simulator)
 - Python (LLM integration and query processing)
- **Database**
 - SQLite / PostgreSQL
- **Natural Language Processing**
 - Open-source language model for Text-to-SQL conversion
- **Frontend**
 - Command line interface or lightweight web interface
- **Integration**
 - File-based communication between Python modules and C++ simulator

Expected Outcomes

- Natural language to SQL query interface.
- SQL validation pipeline ensuring correctness.
- Buffer manager simulation framework.
- Implementation of LRU, MRU, and CLOCK algorithms.
- Experimental evaluation of buffer replacement strategies.
- Comparative analysis using metrics such as:
 - Disk read count
 - Buffer hit ratio
 - Page fault frequency

Project Timeline

Week	LLM / Interface Development	Buffer Manager Simulation (C++)
Week 1	• Design natural language query interface • Select and setup Text-to-SQL model • Define relational database schema	• Study buffer manager architecture • Design buffer pool data structures • Define page access simulation format
Week 2	• Implement Natural Language $\rightarrow$ SQL translation • Develop SQL validation pipeline • Connect frontend interface with query module	• Implement buffer pool simulator • Implement page access input handling • Implement disk read and page fault tracking
Week 3	• Execute validated SQL queries • Generate page access traces • Integrate pipeline with simulator input	• Implement replacement algorithms: LRU, MRU, CLOCK • Implement performance metrics collection
Week 4	• Integrate full query pipeline • Natural Language $\rightarrow$ SQL $\rightarrow$ Page Trace • Improve query validation robustness	• Run experiments with query workloads • Analyze disk I/O statistics • Compare buffer replacement strategies
Final Week	• Refine full system integration • Optimize implementation and remove bottlenecks • Prepare documentation, results, and visualizations • Prepare final report and project demonstration	